SentinelGW

API Security Gateway and Threat Monitoring Platform

Document type: Project proposal and implementation plan
Project status: Planning phase
Proposed technology: React, TypeScript, Tailwind CSS, FastAPI, Python, SQLModel, SQLite, and WebSockets

---

1. Executive Summary

SentinelGW is a proposed API Security Gateway and real-time threat monitoring platform.

The platform will sit between API clients and backend applications. Instead of clients calling a backend service directly, they will call SentinelGW. SentinelGW will inspect each request, apply security policies, block or rate-limit malicious activity, forward approved requests to the correct backend, and record the result for monitoring and investigation.

The project will support multiple API projects. Each connected backend will have its own project context, imported routes, request counters, alerts, analytics, rate limits, and attack-test results. This separation will prevent demo data or traffic from one application from being confused with another application.

SentinelGW will be designed around the OWASP API Security Top 10 and will provide:

- Reverse-proxy protection for real API traffic.
- OpenAPI and Swagger-based API onboarding.
- Project-specific route inventories.
- Authentication and authorization validation.
- Rate limiting and burst protection.
- Injection, SSRF, mass-assignment, and shadow API detection.
- Response hardening and sensitive-field redaction.
- Live traffic monitoring through WebSockets.
- Threat alerts, investigation workflows, and containment actions.
- Adaptive attack simulations that target the currently selected API project.
- Role-based access for administrators, analysts, developers, and viewers.

The Attack Simulator will be a validation tool, not the main purpose of the application. The main purpose will be to protect and observe real API applications.

---

2. Problem Statement

Modern applications often expose multiple APIs across local development environments, internal systems, cloud deployments, and third-party integrations. These APIs may have inconsistent authentication, undocumented routes, weak rate limits, or vulnerabilities that are difficult to observe centrally.

Common operational problems include:

- Clients can bypass security tooling by calling backend services directly.
- Teams do not have a central inventory of API routes.
- OpenAPI specifications are available but are not connected to runtime security controls.
- Security teams cannot easily distinguish traffic belonging to different applications.
- Threat events are spread across application logs and are difficult to investigate.
- Developers lack a safe way to verify whether API security controls work.
- Dashboards may combine demo, test, and production-like data, causing confusion.
- Security policies are difficult to test before enforcement.

SentinelGW will address these problems by creating one controlled entry point for API traffic and one project-aware control plane for monitoring and response.

---

3. Project Objective

The objective is to design and implement a usable API gateway that can connect to any supported backend API, protect its traffic, and provide project-specific security visibility.

The platform will allow a user to:

1. Connect a local or deployed backend API.
2. Import its OpenAPI specification and route definitions.
3. Send API traffic through SentinelGW.
4. Apply security checks before traffic reaches the backend.
5. Monitor allowed, blocked, and rate-limited requests in real time.
6. Investigate alerts and contain malicious clients.
7. Run controlled security tests against the selected API project.
8. View dashboards and analytics for only the project currently being observed.

---

4. Proposed System Concept

The expected traffic flow will be:

```
API Client, Web App, Mobile App, Swagger, Postman, or cURL
                              |
                              v
              SentinelGW protected endpoint
                    http://host:8000/gw/*
                              |
             Security inspection and policy checks
                              |
               Allowed, blocked, or rate-limited
                              |
                              v
               Selected backend API application
```

For multiple projects, SentinelGW will also support a project slug:

```
/gw/{project-slug}/{backend-path}
```

Examples:

```
/gw/parking-api/parking/availability
/gw/order-service/orders/5001
/gw/customer-api/users/101
```

Calling the backend directly will bypass SentinelGW. Therefore, applications and API clients must be configured to use the gateway URL when protection and monitoring are required.

---

5. Target Users and Responsibilities

5.1 Security Administrator

The administrator will configure gateway policies, manage connected projects, control enforcement modes, manage users, configure authentication settings, and maintain the blocklist.

5.2 Security Analyst

The analyst will monitor live requests, investigate alerts, review evidence, update incident status, block malicious clients, and generate triage guidance.

5.3 API Developer or Service Owner

The developer will connect backend projects, import OpenAPI specifications, review registered and shadow routes, test JWT behavior, tune route limits, and validate policies.

5.4 Compliance Viewer

The viewer will have read-only access to dashboards, OWASP coverage, analytics, and reports.

5.5 External API Consumer

External clients will not use the management dashboard. They will call protected `/gw/*` endpoints using JWTs, API keys, or public access according to route policy.

---

6. Functional Scope

6.1 Multi-Project API Onboarding

The platform will allow users to connect backend APIs using:

- Local URLs such as `http://127.0.0.1:8001`.
- Any development port such as `2340`, `3000`, or `8080`.
- Deployed HTTP or HTTPS API URLs.
- A Swagger UI URL.
- An OpenAPI JSON or YAML document.

When a backend is connected, SentinelGW will:

- Create a project with a clear title.
- Generate a unique project slug.
- Store the backend base URL.
- Attempt to retrieve `/openapi.json`.
- Import supported HTTP routes.
- Infer common security metadata from paths and OpenAPI operations.
- Create a separate project tab.
- allow the project to become the default `/gw/*` target.

6.2 Project Separation

Each API project will have an isolated logical view containing:

- Registered routes.
- Shadow routes.
- Route request counters.

- Traffic logs.
- Alerts.
- Dashboard metrics.
- Analytics.
- Rate-limit configuration.
- Attack simulation results.

A global **Observing Project** selector will define which project the user is currently viewing.

Changing this selection will update all project-aware pages without mixing information from other projects.

6.3 Reverse Proxy Gateway

The FastAPI backend will expose a catch-all gateway endpoint:

```
/gw/{path}
```

It will also support explicit project routing:

```
/gw/{project-slug}/{path}
```

Approved requests will be forwarded asynchronously to the selected upstream backend. Response status, body, and allowed headers will be returned to the client after response security processing.

6.4 Swagger and OpenAPI Proxying

SentinelGW will proxy:

- `/gw/docs`
- `/gw/redoc`
- `/gw/openapi.json`
- Project-slug equivalents of these paths

The gateway will rewrite the OpenAPI `servers` value and Swagger specification URL so that Swagger **Try it out** requests are sent through SentinelGW rather than directly to the backend.

This will make Swagger-based tests visible in traffic logs and request counters.

6.5 Request Security Pipeline

Each request will pass through a planned multi-stage pipeline.

Stage 1: Request Context

- Generate or preserve a request ID.
- Capture source IP and user agent.
- Record request size and timing.
- Resolve the selected upstream project.

Stage 2: Gateway Mode

- Enforce security when protection is enabled.
- Support controlled bypass mode for demonstrations and comparisons.
- Continue recording requests even in bypass mode.

Stage 3: Blocklist

- Reject blocked IP addresses.
- Reject blocked authenticated users.
- Reject revoked or blocked API keys.

Stage 4: Route Inventory

- Match the request to an imported route pattern.
- Increment the route request counter.
- Discover undocumented shadow endpoints.
- Apply the shadow API policy.

Stage 5: Authentication

- Validate JWT algorithm restrictions.
- Reject unsigned `alg: none` tokens.
- Verify signatures.
- Validate expiration and time claims.
- Validate issuer and audience.
- Check token revocation identifiers.
- Validate API keys when configured.

Stage 6: Authorization

- Enforce administrator-only functions.
- Compare resource identifiers with the authenticated principal for BOLA protection.
- Validate route-specific roles and scopes.

Stage 7: Rate and Resource Controls

- Apply sustained requests-per-minute limits.
- Apply token-bucket burst limits.

- Return `429 Too Many Requests`.
- Return rate-limit and retry headers.
- Isolate limiter state by API project.

Stage 8: Payload Inspection

- Detect SQL injection.
- Detect NoSQL injection operators.
- Detect cross-site scripting payloads.
- Detect command injection.
- Detect path traversal.
- Detect SSRF attempts against private, loopback, or metadata addresses.
- Detect mass-assignment fields.
- Reject excessive pagination and oversized payloads.

Stage 9: Upstream Forwarding and Response Hardening

- Forward approved requests to the backend.
- Remove unnecessary server-identification headers.
- Add security headers.
- Redact configured sensitive JSON fields.
- Prevent stack-trace leakage where possible.

Stage 10: Logging and Detection

- Store a structured request log.
- Calculate threat and anomaly scores.
- Evaluate behavioral rules.
- Create or update alerts.
- Auto-block critical offenders when enabled.
- Broadcast traffic and alerts through WebSockets.

6.6 Live Request Monitoring

The Live Requests page will show:

- Request time.
- Project context.
- Decision.
- HTTP method and route.
- Client IP and authenticated principal.
- Response status.
- Gateway latency.
- OWASP category.
- Threat score.
- Block reason.

The page will support filtering and request-detail inspection.

6.7 Project-Specific Dashboard

The dashboard will only display data for the selected project.

Planned metrics include:

- Total gateway requests.
- Allowed requests.
- Blocked and rate-limited requests.
- Block rate.
- Open alerts.
- Critical and high alerts.
- Average gateway latency.
- Error rate.
- Unique clients.
- Most active endpoints.
- Recent threats.
- Top offending IP addresses.

The dashboard will update through WebSocket events and periodic refreshes.

6.8 Threat Detection and Alerts

The detection engine will evaluate both individual signatures and repeated behavior, including:

- JWT tampering.
- Expired or revoked token reuse.
- Brute-force login attempts.
- Credential stuffing.
- Object identifier enumeration.
- Privilege escalation attempts.
- SSRF.
- Injection attacks.
- Mass assignment.

- Sensitive business-flow abuse.
- Resource exhaustion.
- Shadow API probing.
- Abnormal request velocity.

Alerts will be scoped to the project that generated the underlying requests.

6.9 Incident Investigation and Response

An alert workbench will provide:

- Alert severity and OWASP classification.
- Related request timeline.
- Request and response evidence.
- Offender profile.
- Threat score.
- AI-assisted or rules-based triage summary.
- Status transitions.
- Analyst notes.
- IP blocking.
- User blocking.
- Token revocation.
- False-positive handling.

6.10 Adaptive Attack Simulator

The simulator will run controlled security-validation requests against the currently selected project.

It will not remain tied to hard-coded demo paths. Instead, it will examine the selected project's imported route inventory and choose the most appropriate route for each test.

Planned scenarios include:

- BOLA object enumeration.
- Credential stuffing and brute force.
- JWT algorithm and signature tampering.
- Excessive data exposure and mass assignment.
- Rate-limit and resource-consumption abuse.
- Broken function-level authorization.
- SSRF against cloud metadata and loopback addresses.
- SQL injection and shadow endpoint probing.

Every simulator request will use:

```
/gw/{selected-project-slug}/...
```

Results will clearly distinguish:

- **Blocked by SentinelGW**
- **Reached the upstream backend**

If a project does not contain a route suitable for a route-specific test, the final implementation should mark that test as not applicable rather than presenting a misleading failure.

6.11 Rate-Limit Testing

The rate-limit page will:

- List only routes for the selected project.
- Allow route-specific requests-per-minute and burst settings.
- Generate a controlled request burst through the selected project's slug.
- Show allowed and throttled requests.
- Display rate-limit headers.
- Show active client bucket utilization.

6.12 API Inventory and Shadow Routes

The inventory will provide:

- Separate tabs for connected API projects.
- Project titles and backend URLs.
- Imported route counts.
- Project request totals.
- Live route counters.
- Registered route details.
- Shadow route discovery.
- Route approval.
- Route blocking.
- OpenAPI import into the selected project.
- Manual route creation.

The built-in demo API will be visually separated from user-connected projects.

6.13 Role-Based Access Control

The management interface will support:

- Admin
- Analyst
- Developer
- Viewer

The navbar will use a compact role dropdown for demonstrations. Production authentication will use authenticated accounts and server-side authorization rather than relying on the demonstration switcher.

6.14 Reports and Governance

The reporting area will provide:

- Executive security metrics.
- OWASP category summaries.
- Alert severity summaries.
- Traffic CSV export.
- Alert CSV export.
- Printable reports.
- Project and time-range filters.

---

7. Planned User Experience

7.1 Navigation Model

The interface will be grouped into three understandable sections.

Monitor Traffic

- Home Dashboard
- Live Requests
- Security Alerts
- Speed and Trends

Connect and Protect APIs

- Connected APIs and Routes
- Security Rules
- Rate Limits
- Login and JWT Keys
- Blocked IPs

Test and Export

- Attack Simulator
- OWASP Guide
- Reports
- Team and Roles

7.2 Global Controls

The top navigation will include:

- Observing Project dropdown.
- Protection status.
- Time-range selector.
- Compact View As role dropdown.

The selected project will be persisted locally so that navigation between pages retains the same context.

7.3 Error Handling

Frontend API operations will use `async/await` and `try/catch`.

Errors returned by FastAPI will be converted into readable messages. Validation arrays and structured objects must not appear as `[object Object]`.

---

8. Proposed Technical Architecture

8.1 Frontend

The frontend will use:

- React.
- TypeScript.
- Vite.
- Tailwind CSS.
- Zustand for application state.
- React Router for navigation.
- Recharts for charts.
- WebSocket connectivity for live updates.

Key frontend responsibilities will include:

- Project selection.
- Role-aware controls.
- Real-time traffic display.
- Alert workflows.
- API onboarding.
- Attack-test execution.
- Dashboard visualization.

8.2 Backend

The backend will use:

- FastAPI.
- Python.
- SQLModel.
- HTTPX for asynchronous proxy requests.
- WebSockets for live telemetry.
- Pydantic for request validation.

Key backend responsibilities will include:

- Reverse proxying.
- Security enforcement.
- Upstream and route management.
- OpenAPI import.
- Logging.
- Threat detection.
- Alert management.
- Rate limiting.
- Reporting APIs.

8.3 Data Storage

The initial development version will use SQLite.

Primary entities will include:

- User.
- Upstream project.
- Route.
- Policy.
- API key.
- Request log.
- Alert.
- Blocklist entry.
- Revoked token.
- Client anomaly baseline.

SQLite is suitable for the planned prototype and demonstration phase. A production deployment should evaluate PostgreSQL for concurrent access, reliability, migrations, retention, and reporting.

8.4 Runtime State

The prototype may use in-memory state for:

- Token-bucket counters.
- Sliding-window rate counters.
- Behavioral detection windows.
- WebSocket connections.

For production, this state should move to a shared system such as Redis so multiple gateway instances can enforce consistent limits.

---

9. Key Management API Areas

The proposed management API will include:

- `/api/v1/inventory` for projects and routes.
- `/api/v1/inventory/sync-swagger-url` for backend connection and OpenAPI discovery.
- `/api/v1/inventory/import-openapi` for pasted specifications.
- `/api/v1/overview` for project-filtered dashboard data.
- `/api/v1/traffic` for project-filtered request logs.
- `/api/v1/alerts` for project-filtered alerts.
- `/api/v1/analytics` for project-filtered trends.
- `/api/v1/ratelimits` for project route limits.
- `/api/v1/simulator` for adaptive security validation.
- `/api/v1/policies` for protection configuration.
- `/api/v1/blocklist` for containment.
- `/ws/live` for real-time events.

All project-aware endpoints will receive or resolve an upstream project identifier.

10. Implementation Plan

Phase 1: Foundation and Project Setup

Planned work:

- Create FastAPI and React applications.
- Configure TypeScript, Tailwind CSS, routing, and state management.
- Configure SQLModel and SQLite.
- Define backend configuration.
- Add health checks.
- Add Docker development setup.

Deliverable:

- Runnable frontend and backend skeleton.

Phase 2: Identity, Roles, and Application Shell

Planned work:

- Create user and role models.
- Implement login and authorization dependencies.
- Build the main layout and navigation.
- Add role-aware controls.
- Add the compact role dropdown for demonstrations.

Deliverable:

- Authenticated management shell with four roles.

Phase 3: Multi-Project API Inventory

Planned work:

- Define upstream and route models.
- Add backend connection workflow.
- Add project titles and unique slugs.
- Add project tabs.
- Add the global Observing Project selector.
- Persist the selected project.
- Separate demo and imported routes.

Deliverable:

- Users can connect and switch between multiple API projects without route mixing.

Phase 4: OpenAPI and Swagger Integration

Planned work:

- Fetch `/openapi.json` from connected backends.
- Parse JSON and YAML specifications.
- Import supported operations.
- Infer authentication, administrator routes, path identifiers, and sensitive flows.
- Add pasted-spec import.
- Add readable validation errors.
- Proxy and rewrite Swagger UI.

Deliverable:

- Imported routes are assigned to the correct project and Swagger requests pass through the gateway.

Phase 5: Reverse Proxy and Security Pipeline

Planned work:

- Implement catch-all gateway routing.
- Resolve active and slug-specific projects.
- Add request correlation.
- Add blocklist checks.
- Add JWT and API-key validation.
- Add BOLA and BFLA controls.
- Add rate limiting.
- Add payload inspection.
- Forward approved requests.
- Harden responses.

Deliverable:

- Real client traffic can be protected and forwarded to connected backends.

Phase 6: Logging, Detection, and WebSockets

Planned work:

- Store request logs with project IDs.
- Add threat scoring.

- Add behavioral detection.
- Isolate rate and detection state by project.
- Create and deduplicate project-scoped alerts.
- Broadcast project IDs with WebSocket events.

Deliverable:

- Every gateway request produces project-aware telemetry and threat events.

Phase 7: Monitoring and Incident Response

Planned work:

- Build the project-filtered dashboard.
- Build live traffic monitoring.
- Build alerts and incident detail pages.
- Add request evidence drawers.
- Add containment actions.
- Add analytics and anomaly displays.

Deliverable:

- Security teams can observe and investigate one project at a time.

Phase 8: Adaptive Testing and Validation

Planned work:

- Build the attack scenario catalogue.
- Resolve target routes from the selected project's inventory.
- Use the selected project slug for every test.
- Add project-specific simulated source IPs.
- Distinguish gateway blocks from upstream errors.
- Add adaptive rate-limit tests.
- Mark non-applicable scenarios clearly.

Deliverable:

- Security tests validate the currently observed project rather than a hard-coded demo API.

Phase 9: Reporting, Documentation, and Usability

Planned work:

- Add project-filtered reports.
- Add CSV exports.
- Add OWASP coverage guidance.
- Add role-based operational documentation.
- Simplify labels and navigation.
- Ensure all frontend requests use `async/await` with `try/catch`.

Deliverable:

- A presentable and understandable management experience.

Phase 10: Production Hardening

Planned work:

- Replace demonstration secrets.
- Add secure key storage and rotation.
- Replace permissive CORS.
- Add TLS and trusted proxy configuration.
- Add database migrations.
- Evaluate PostgreSQL and Redis.
- Add audit logging.
- Add test coverage and load testing.
- Add container health checks.
- Define log retention.
- Add deployment and rollback procedures.

Deliverable:

- A deployment candidate suitable for controlled staging evaluation.

---

11. Testing Strategy

11.1 Unit Testing

Unit tests will cover:

- Route matching.
- JWT validation.
- BOLA ownership checks.
- BFLA role checks.
- Signature detection.

- SSRF URL checks.
- OpenAPI parsing.
- Rate-limit calculations.
- Project filtering.

11.2 Integration Testing

Integration tests will verify:

- Client to gateway to upstream traffic.
- Project slug routing.
- Swagger request rewriting.
- Logging and alert creation.
- Dashboard filtering.
- WebSocket broadcasts.
- Blocklist enforcement.
- Response redaction.

11.3 Security Validation

Security tests will verify:

- Invalid JWT rejection.
- Object-access violations.
- Administrator route protection.
- Injection blocking.
- SSRF blocking.
- Mass-assignment blocking.
- Rate-limit enforcement.
- Shadow route handling.

11.4 Frontend Testing

Frontend testing will verify:

- Project switching.
- Route separation.
- Role-aware controls.
- Error messages.
- Live counter updates.
- Project-filtered dashboards.
- Adaptive simulator targeting.

11.5 Performance Testing

Performance testing will measure:

- Gateway latency overhead.
- Requests per second.
- WebSocket event throughput.
- Database write performance.
- Rate-limiter consistency.
- Behavior under large route inventories.

---

12. Acceptance Criteria

The planned release will be accepted when:

1. A user can connect a backend on an arbitrary local port or deployed URL.
2. The backend appears as a titled project.
3. Imported routes appear only in that project.
4. Swagger Try it out requests can pass through SentinelGW.
5. cURL, Postman, frontend, and mobile clients can call protected routes.
6. Every gateway request is logged under the correct project.
7. Route counters update after a request.
8. The dashboard displays only the selected project's data.
9. Live traffic and alerts display only the selected project's events.
10. Rate-limit settings and tests apply to the selected project.
11. Attack Simulator requests use the selected project slug.
12. Simulator results distinguish gateway enforcement from upstream responses.
13. Common OWASP attack payloads are blocked or clearly reported.
14. Role permissions prevent unauthorized management changes.
15. Errors are readable and do not display `[object Object]`.
16. The application can be started locally and with Docker.

---

13. Non-Functional Requirements

Security

- Management mutations must require authorization.
- Secrets must not be stored in source code for production.
- Sensitive headers and fields must be masked in logs.
- Gateway-generated security decisions must be traceable.

Performance

- Security checks should add minimal latency.
- Long-running detection work should not block proxy responses unnecessarily.
- Traffic pages should remain responsive during high event volume.

Reliability

- A failed WebSocket connection should reconnect automatically.
- A failed upstream should return a controlled `502 Bad Gateway`.
- An unavailable OpenAPI document should not prevent a backend project from being registered.

Usability

- Users should always know which project they are observing.
- Demo routes and imported routes must remain visually separated.
- Buttons and labels should use plain language.
- Destructive actions should be explicit.

Maintainability

- Security stages should be modular.
- API schemas should have stable names.
- Frontend and backend contracts should be tested.
- Database changes should use migrations before production.

---

14. Risks and Mitigations

Risk: Clients bypass the gateway

Mitigation:

- Configure client base URLs to use SentinelGW.
- Restrict direct backend network access in deployed environments.
- Document that direct backend calls cannot be logged or protected by the gateway.

Risk: Imported specifications are incomplete

Mitigation:

- Support runtime shadow route discovery.
- Allow manual route creation.
- Display unknown routes for review.

Risk: Generic authorization inference causes false positives

Mitigation:

- Allow developers to review inferred BOLA, BFLA, and authentication settings.
- Support Monitor mode before enforcement.
- Mark simulator scenarios as not applicable when required route semantics are unavailable.

Risk: In-memory state is inconsistent across instances

Mitigation:

- Use Redis in multi-instance production deployments.

Risk: SQLite does not support production traffic volume

Mitigation:

- Use SQLite for local development and demonstrations.
- Plan migration to PostgreSQL for staging and production.

Risk: The gateway becomes a bottleneck

Mitigation:

- Use asynchronous I/O.
- Add load tests.
- Scale stateless gateway instances.
- Move shared counters to Redis.

Risk: Attack simulation affects real data

Mitigation:

- Clearly label the simulator.
- Require authorization.

- Prefer staging or dedicated test projects.
- Add confirmation before testing non-demo deployments.
- Use test payload markers and configurable safe-test modes.

---

15. Out of Scope for the Initial Prototype

The first planned release will not attempt to provide:

- Full distributed denial-of-service protection at network scale.
- Replacement for a cloud load balancer or CDN.
- Automatic discovery of traffic that does not pass through SentinelGW.
- Complete business-level authorization without application-specific ownership data.
- Guaranteed vulnerability scanning of every possible backend technology.
- Multi-region high availability.
- Enterprise SIEM integration unless added as a later milestone.

---

16. Proposed Delivery Outputs

The project will produce:

- FastAPI gateway and management backend.
- React management dashboard.
- SQLite development database.
- Multi-project API inventory.
- OpenAPI import workflow.
- Reverse-proxy security pipeline.
- Project-aware request logs and alerts.
- Adaptive attack simulator.
- Vulnerable demo API for controlled testing.
- Docker development configuration.
- Role-based user guide.
- Verification and demonstration scripts.

---

17. Success Measures

The project will be considered successful when it demonstrates that:

- Real requests can be routed through the gateway.
- Malicious requests are stopped before reaching the backend.
- Safe requests reach the correct backend.
- Multiple API projects remain logically separated.
- Users can understand which project they are viewing.
- Security events appear in near real time.
- Developers can import routes without manual duplication.
- Security controls can be validated through controlled tests.
- Stakeholders can understand the platform's value from the dashboard and reports.

---

18. Planned Demonstration

The planned demonstration will follow this sequence:

1. Explain the client → SentinelGW → backend architecture.
2. Connect a backend API using its URL.
3. Show the automatically created project tab.
4. Import or synchronize OpenAPI routes.
5. Open the gateway-proxied Swagger UI.
6. Execute a normal request.
7. Show the project route counter and dashboard update.
8. Run an SSRF or JWT tampering test against the selected project.
9. Show that SentinelGW blocked the request.
10. Open Live Requests and inspect the event.
11. Open the generated project-scoped alert.
12. Switch to another project and show that its dashboard remains separate.

This demonstration will communicate the core value clearly: SentinelGW is a gateway for protecting real APIs, while the simulator is a tool for proving that the protection works.

---

19. Conclusion

SentinelGW is proposed as a centralized security and observability layer for API applications.

Its main value will come from combining:

- Real reverse-proxy enforcement.
- Multi-project API onboarding.

- OWASP-aligned security controls.
- Live traffic and alert monitoring.
- Project-specific dashboards.
- Controlled, adaptive security testing.
- Clear role-based workflows.

The proposed phased plan will allow the team to deliver a demonstrable prototype first, validate the user experience and security model, and then progress toward a hardened staging and production architecture.